

# Terms of Reference Coverage

---

## Wheat Water Productivity Dashboard

Ethiopian Institute of Agricultural Research · IWMI East Africa · FAO WaPOR Phase II  
Generated 2026-08-19

---

## ToR coverage

---

How this build maps to `ToR_Dashboard_Developer.docx`, and — as importantly — what it does not yet cover. Nothing below is inferred from the code; each row says where the work lives or why it is out of scope for a software build.

### A note on objective 4

The ToR was written expecting a LightGBM model as the analytical core, and an earlier revision of this build shipped one — trained on synthetic samples, as placeholder scaffolding awaiting the model IWMI would deliver.

The delivered reference implementation, `ETH_WWP_WaPORv3_Irrigaed_Wheat_2026_Final.ipynb`, turns out to contain **no machine learning**. It is a deterministic FAO biomass-to-yield calculation over two WaPOR v3 variables. Porting it, as the consultancy scope requires, therefore retires the prediction service rather than filling it in: `model_service.py`, `train_model.py`, `backend/models/` and the `/api/model/*` routes have been removed, along with the LightGBM dependency.

This is a better outcome than it may read as. The tool now shows a user the complete derivation of any value — every measured input, every parameter, every intermediate — which is a stronger form of the explainability objective 4 asked for than a SHAP waterfall over a model trained on data nobody has yet collected. The trade-off is real and worth stating: the socioeconomic variables the ToR lists (fertilizer, improved seed, extension visits, market distance) no longer enter the estimate at all, because the reference method does not use them. If EIAR wants those drivers modelled, that is new scope on top of the reference method, not a restoration of the placeholder.

## Objectives

| # | OBJECTIVE                                                   | STATUS                                                                                                                                                                                                        |
|---|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Design frontend, backend, inference and integration layers  | Done — the <a href="#">Architecture</a> document                                                                                                                                                              |
| 2 | Interactive dashboard with multiple data-entry mechanisms   | Done — admin unit, shapefile/GeoJSON upload, draw-on-map, and per-feature scheme files in the reference notebook's own format, including the 2026 campaign shapefiles themselves where the machine holds them |
| 3 | Python analytical engine as a scalable backend service      | Done — <code>backend/app/wwpt.py</code> + <code>engine.py</code> behind <code>POST /api/analysis</code>                                                                                                       |
| 4 | Analytical model deployed as a prediction service           | Done, but <b>not as LightGBM</b> — see the note above. The reference method is served at <code>/api/predict</code> and <code>/api/explain</code> , with parity to the notebook asserted in CI.                |
| 5 | Scalable spatial database for inputs, artifacts and outputs | <b>Partial</b> — upload registry and run store are in-memory. See below.                                                                                                                                      |
| 6 | Integration into the EIAR web platform                      | Build-side done — relative-base bundle, sub-path safe, EIAR identity applied. Coordination with EIAR IT/GIS is an engagement activity.                                                                        |
| 7 | System testing and quality assurance                        | Done — 124 automated checks across <code>tests/</code>                                                                                                                                                        |
| 8 | Technical documentation                                     | Done — README, ARCHITECTURE, generated OpenAPI at <code>/docs</code>                                                                                                                                          |
| 9 | Train EIAR staff                                            | Out of scope for a software build — delivery activity                                                                                                                                                         |

## Work packages

**WP1 — Inception, requirements, architecture.** Architecture document produced. The approved `WWP Dashboard.html` prototype served as the click-through prototype and this build implements its user journeys; two visual decisions depart from it deliberately and are justified in [the Architecture document § Visualization palette](#). Reviewing the live EIAR platform for integration constraints needs access to that platform and remains open.

**WP2 — Frontend and dashboard.** React 18 + Vite, Leaflet 1.9 for the map, charts as inline SVG. All three AOI journeys implemented, plus the per-scheme workflow: a shapefile or GeoJSON carrying ID/SOS/EOS per feature (and Location for sample points) is estimated feature by feature over each feature's own season, and the results are laid out as the reference notebook lays them out — the per-feature table, the per-plot medians grouped as notebook cell 26 groups them, and the two figures it plots — with both tables downloadable in the notebook's columns. Validation runs on both sides: the client constrains selectors so an invalid combination is hard to express, and the server independently rejects unknown admin units, invalid season/system pairs, unknown years, degenerate polygons, non-EPSG:4326 shapefiles, boundaries outside Ethiopia, oversized extents, corrupt archives and unsupported file types — each with a message aimed at the user. EIAR colours, typography and logo mark are applied throughout, and the layout stacks cleanly from 1600 px to 420 px.

**WP3 — Backend, API, analytical tool integration.** Eleven REST routes with generated OpenAPI documentation. Response caching (30-minute TTL, keyed on the canonical request) and `Idempotency-Key` support for retrievable POSTs are both implemented and tested. The WaPOR data-access layer sits behind a provider interface: a synthetic provider ships so the pipeline runs end-to-end offline, and `wapor.py` implements live FAO WaPOR v3 retrieval, selected by `WWP_PROVIDER`.

**WP4 — Analytical model integration and deployment.** The reference notebook's method is ported to `backend/app/wwpt.py`: the biomass-to-yield equations, the crop parameters and the SOS/EOS season handling, with an input validator mirroring `etwapor.util.validate_input`. **Parity with the notebook is asserted, not assumed** — `tests/test_notebook_parity.py` replays all 16 results the notebook publishes and requires the same yield and water productivity at its own precision. That check also gates the Docker build, so an image whose estimates disagree with the reference cannot be produced. Inference accepts both a point and an area of interest. Explanation is the full derivation chain, rendered so a reader can check the arithmetic by hand; the e2e and UI suites both verify that the numbers shown actually reproduce the answer shown.

Crop parameters are configurable through `WWP_CROP_PARAMS` rather than hard-coded, which is the mechanism by which EIAR's agro-ecology-specific values — the improvement the notebook itself calls for — get adopted without a code change.

**WP5 — Website integration.** The bundle builds with a relative base so it deploys under any sub-path behind a reverse proxy, and the backend can serve it directly. A `Dockerfile` and `railway.json` package the whole application for container hosting — see [the Deployment Guide](#). All five required supporting pages are written and reachable from the header and footer: methodology, data sources, user guide, how to cite, and disclaimer. **Web analytics (GA4) is not wired up** — it needs EIAR's property ID and a decision on consent handling, so it is left as a one-file addition rather than a guess.

**WP6 — Documentation, training, support.** Architecture, API and deployment documentation are in place. Training delivery and the three-month post-deployment support window are engagement activities.

## What deployment still needs

Honest list of the gaps between this build and a production system on EIAR infrastructure:

1. **Turn on the WaPOR provider and prove a pixel read.** `wapor.py` is written and its catalogue contract is verified against the live FAO service by `/api/wapor/check` and the e2e suite. What has not run is the GeoTIFF sampling itself, which needs `rasterio` (commented out in `requirements.txt` because GDAL is a heavy dependency). Until `WWP_PROVIDER=wapor` is set and that check is green, every number in the dashboard is synthetic — plausible and internally consistent, but not measured.

One thing to expect when it is turned on: the seasonal NPP and AETI will not match the figures the reference notebook printed. `etwapor.download` builds every dekadal filename with a literal `D1`, so all three dekads of a month resolve to the month's first raster while their day weights are applied in full; `wapor.py` requests the dekad it is summing. The three rasters are distinct and all published, so the divergence is a defect in the reference, not in this service, and the arithmetic downstream of those two numbers is identical — that is what the parity suite shows. Raise it with IWMI before the campaign figures are published; `docs/ARCHITECTURE.md` carries the evidence.

2. **Crop parameters validated for Ethiopia.** The values in use are general FAO reference values that reproduce the reference notebook. Neither they nor the notebook have been validated against field-measured Ethiopian yields. This is the single largest source of uncertainty in the output, and the notebook says so itself. EIAR trial data should replace them via `WWP_CROP_PARAMS`.
3. **Per-plot SOS and EOS, in the raster journey.** Scheme files already carry their own growing seasons and are estimated over them, feature by feature ( `schemes.py` ). What still uses one window for everything is the area-of-interest journey, where the season comes from the selector because a continuous extent has no per-feature dates to read. Letting a drawn or administrative AOI take an explicit SOS/EOS pair is a small extension — `wwpt.season_window` is the only thing between the selector and the provider — but it is not built.
4. **PostGIS for the spatial database.** Objective 5 is only partly met. The upload registry ( `aoi._UPLOADS` ) and completed-run store ( `engine.RUNS` ) are in-memory dictionaries, so a restart drops uploaded boundaries and CSV export links, and they cannot be shared across workers. Both are small, well-isolated modules to move to PostGIS tables, and the CSA boundary layer belongs there too.
5. **Redis for the cache** if the service runs more than one worker — `cache.TTLCache` is per-process, so with several workers the hit rate degrades and idempotency keys stop being reliable. This matters more with the WaPOR provider enabled, where a cache miss is expensive rather than instant.
6. **GA4 analytics**, pending EIAR's property ID and consent approach.
7. **Amharic localization.** The language toggle is present but reports that the Amharic interface is planned; the UI strings are not yet externalized.
8. **Authentication**, if EIAR wants the analysis routes restricted. Every route is currently open, which suits a public research tool but would need a gateway otherwise. Note that the admin-token guard has gone with the model-management routes it protected — there is no longer any state-changing route to protect.
9. **Map PNG export.** CSV export is implemented; the prototype's "Save map (PNG)" button is not, since a faithful map export needs server-side tile composition. The button was removed rather than left as a control that does nothing.

## Test coverage

| SUITE                         | CHECKS           | COVERS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tests/test_notebook_parity.py | 16 records + LGP | Every result the reference notebook publishes, reproduced at its own precision. No server, no network.                                                                                                                                                                                                                                                                                                                                                                                                                |
| tests/test_api_e2e.py         | 56               | All three AOI journeys, caching, idempotency, point prediction, derivation arithmetic, CSV export in the notebook's column names, the live WaPOR catalogue contract, and 10 validation/error paths                                                                                                                                                                                                                                                                                                                    |
| tests/test_campaign_files.py  | 30               | Ingest parity on the real 2026 campaign shapefiles: LGP per feature, the 57 sample points collapsing to the notebook's six plots, and the in-app loader producing feature for feature what uploading the zipped shapefile produces. Skips unless the field data is present, which is not committed ( <code>WWP_CAMPAIGN_DATA</code> , default <code>./Data</code> )                                                                                                                                                   |
| tests/test_schemes.py         | 44               | The per-scheme workflow: attribute parsing, per-feature estimation over each feature's own season, notebook column names and order, the per-plot median grouping, both CSV exports, and the validation and error paths                                                                                                                                                                                                                                                                                                |
| tests/test_ui.mjs             | 67               | Rendering, chart integrity, label overflow, table views, results-panel open/close and map re-sync, pixel inspect, the rendered derivation reproducing the rendered result, both polygon-finish paths, content pages, four viewport widths, console cleanliness, and the per-scheme workflow end to end, driven from the ready-made-file list — the real 57-sample campaign file where the machine has it, the generated sample otherwise — so an unreachable workflow fails the suite even when the API still answers |

All 213 pass against the current build. The API suite's WaPOR catalogue checks run against the live FAO service and are skipped, not failed, when it is unreachable, so the suite still passes offline.